CROP DISEASE DETECTION APP

Detailed Build Prompt Library

PHASE 1 — Machine Learning & Data Pipeline

Prompts 1–55 · 55 Fully Detailed Prompts

Prepared for: Priyanshu Kumar Gupta & Team
(Sadhana Bag, Kumar Gaurav, Rajdeep Kumar)

Part 1 of 4 — execute this phase fully before opening the next

Phase Overview

This phase produces the two frozen artifacts every other phase depends on: crop_disease_model.keras and class_names.json. Nothing in Phase 2 or 3 should begin until these exist.

Reference: Full Project File Structure

Every prompt in this document references paths from the following master file tree. Keep it visible while executing.

```
crop-disease-detection/
|-- README.md
|-- ml/
|   |-- requirements.txt
|   |-- dataset/
|   |   |-- raw/
|   |   |-- filtered/
|   |   |-- split/
|   |   |   |-- train/<class>/
|   |   |   |-- val/<class>/
|   |   |   `-- test/<class>/
|   |   `-- real_world_sample/<class>/
|   |-- notebooks/
|   |   |-- 00_colab_setup.ipynb
|   |   |-- 01_dataset_exploration.ipynb
|   |   `-- 02_preprocessing_test.ipynb
|   |-- preprocessing/
|   |   |-- loader.py
|   |   |-- augmentation.py
|   |   |-- dataset_pipeline.py
|   |   `-- class_weights.py
|   |-- training/
|   |   |-- baseline_cnn.py
|   |   |-- train_baseline.py
|   |   |-- transfer_mobilenetv2.py
|   |   |-- train_transfer.py
|   |   `-- transfer_resnet50.py
|   |-- evaluation/
|   |   |-- evaluate.py
|   |   |-- confusion_matrix.py
|   |   |-- compare_models.py
|   |   |-- confidence_threshold_experiment.py
|   |   `-- real_world_eval.py
```

```
|   `-- models/
|       |-- crop_disease_model.keras
|       `-- class_names.json
|
|-- backend/
|   |-- requirements.txt
|   |-- .env.example
|   |-- alembic/
|   |-- scripts/
|   |   `-- reset_db.py
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_auth.py
|   |   |-- test_users.py
|   |   |-- test_predict.py
|   |   `-- test_history_feedback.py
|   `-- app/
|       |-- main.py
|       |-- core/
|       |   |-- config.py
|       |   |-- security.py
|       |   `-- dependencies.py
|       |-- db/
|       |   |-- session.py
|       |   `-- seed_diseases.py
|       |-- models/
|       |   |-- user.py
|       |   |-- prediction.py
|       |   |-- disease.py
|       |   `-- feedback.py
|       |-- schemas/
|       |   |-- auth.py
|       |   |-- user.py
|       |   `-- prediction.py
|       |-- routes/
|       |   |-- auth.py
|       |   |-- users.py
|       |   |-- predict.py
|       |   `-- diseases.py
|       |-- services/
|       |   |-- model_service.py
|       |   `-- preprocessing_service.py
|       `-- disease_data/
|
|-- frontend/
|   |-- package.json
|   |-- tailwind.config.js
|   |-- index.html
|   `-- src/
|       |-- App.tsx
|       |-- components/
|       |   |-- ui/
|       |   |   |-- Button.tsx
|       |   |   |-- Card.tsx
|       |   |   |-- Input.tsx
|       |   |   |-- Avatar.tsx
|       |   |   `-- Spinner.tsx
|       |   |-- StatsCard.tsx
|       |   |-- RecentScansTable.tsx
|       |   |-- ProtectedRoute.tsx
|       |   `-- Layout.tsx
|       |-- pages/
|       |   |-- Signup.tsx
|       |   |-- Login.tsx
|       |   |-- Dashboard.tsx
|       |   |-- NewScan.tsx
|       |   |-- Analyzing.tsx
|       |   |-- Result.tsx
```

```
|       |   |-- History.tsx
|       |   |-- Profile.tsx
|       |   `-- About.tsx
|       |-- services/
|       |   `-- api.ts
|       |-- store/
|       |   `-- authStore.ts
|       |-- hooks/
|       `-- assets/
|
`-- docs/
    |-- SETUP.md
    |-- ARCHITECTURE.md
    |-- API_REFERENCE.md
    |-- DATABASE_SCHEMA.md
    |-- SECURITY.md
    |-- PERFORMANCE.md
    |-- TESTING_REPORT.md
    `-- DEMO_SCRIPT.md
```

Detailed Prompts

1.A Environment & Project Setup

PROMPT 1 — Create the root monorepo folder crop-disease-detection/ with subfolders ml/, backend/,...

Objective: Create the root monorepo folder `crop-disease-detection/` with subfolders `ml/`, `backend/`, `frontend/`, `docs/`, and a root `README.md` summarizing the project and linking to each subfolder.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `crop-disease-detection/`, `ml/`, `backend/`, `frontend/`, `docs/`, `README.md`

Detailed Requirements

- Create the root monorepo folder `crop-disease-detection/` with subfolders `ml/`, `backend/`, `frontend/`, `docs/`, and a root `README.md` summarizing the project and linking to each subfolder.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 2 — Inside ml/, create the folder structure

Objective: Inside `ml/`, create the folder structure: `dataset/`, `notebooks/`, `preprocessing/`, `training/`, `evaluation/`, `models/`, and a `requirements.txt` listing tensorflow, keras, opencv-python, pillow, numpy, pandas, matplotlib, scikit-learn.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `ml/`, `dataset/`, `notebooks/`, `preprocessing/`, `training/`, `evaluation/`, `models/`, `requirements.txt`

Detailed Requirements

- Inside `ml/`, create the folder structure: `dataset/`, `notebooks/`, `preprocessing/`, `training/`, `evaluation/`, `models/`, and a `requirements.txt` listing tensorflow, keras, opencv-python, pillow, numpy, pandas, matplotlib, scikit-learn.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 3 — Set up a Python virtual environment (venv) for the ml/ folder and document activation st...

Objective: Set up a Python virtual environment (`venv`) for the `ml/` folder and document activation steps in `ml/README.md`.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `ml/`, `ml/README.md`

Detailed Requirements

- Set up a Python virtual environment (`venv`) for the `ml/` folder and document activation steps in `ml/README.md`.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 4 — Create a Google Colab notebook template ml/notebooks/00_colab_setup.ipynb that mounts Goog...

Objective: Create a Google Colab notebook template `ml/notebooks/00_colab_setup.ipynb` that mounts Google Drive, installs requirements, and verifies GPU availability.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `ml/notebooks/00_colab_setup.ipynb`

Detailed Requirements

- Create a Google Colab notebook template `ml/notebooks/00_colab_setup.ipynb` that mounts Google Drive, installs requirements, and verifies GPU availability.
- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

PROMPT 5 — Initialize a Git repository at the project root, add a .gitignore covering venv/, __pyc...

Objective: Initialize a Git repository at the project root, add a `.gitignore` covering `venv/`, `__pycache__/`, `node_modules/`, `.env`, `dataset/`, and large model files, and make the first commit.

Why This Matters: Project-setup prompts establish the skeleton every later prompt in this module builds on. Getting folder names, config loading, and dependency lists right here prevents painful restructuring later.

Primary File(s): `.gitignore`, `venv/`, `__pycache__/`, `node_modules/`, `.env`, `dataset/`

Detailed Requirements

- Initialize a Git repository at the project root, add a `.gitignore` covering `venv/`, `__pycache__/`, `node_modules/`, `.env`, `dataset/`, and large model files, and make the first commit.

- Use the exact folder/file names given so later prompts (which reference these paths directly) do not break.
- Pin dependency versions in the relevant requirements/package file rather than leaving them unpinned.
- Add a short README or docstring at the top of the new file explaining its purpose.
- Keep secrets/config out of version control; use `.env` / `.env.example` as appropriate.

Technical Specification

No existing schema/API dependency — this is foundational scaffolding. Follow the folder tree in the project's master file-structure reference exactly.

Acceptance Criteria

- ☐ The new folder/file exists at the exact path specified.
- ☐ Running the relevant install command (`pip install -r requirements.txt` / `npm install`) succeeds with no errors.
- ☐ No secrets or credentials are committed to the repository.

Edge Cases & Notes

- If a dependency version conflicts with another already-pinned dependency, document the conflict and choose the lowest version that satisfies both rather than silently upgrading everything.

1.B Dataset Acquisition & Exploration

PROMPT 6 — Write a script ml/dataset/download_dataset.py that downloads the PlantVillage dataset from...

Objective: Write a script `ml/dataset/download_dataset.py` that downloads the PlantVillage dataset from Kaggle via the Kaggle API into `ml/dataset/raw/`.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/download_dataset.py`, `ml/dataset/raw/`

Detailed Requirements

- Write a script `ml/dataset/download_dataset.py` that downloads the PlantVillage dataset from Kaggle via the Kaggle API into `ml/dataset/raw/`.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 7 — Write ml/notebooks/01_dataset_exploration.ipynb that counts total images and images per cl...

Objective: Write `ml/notebooks/01_dataset_exploration.ipynb` that counts total images and images per class folder and prints a summary table.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/notebooks/01_dataset_exploration.ipynb`

Detailed Requirements

- Write `ml/notebooks/01_dataset_exploration.ipynb` that counts total images and images per class folder and prints a summary table.

- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 8 — Extend the exploration notebook to plot a bar chart of class distribution using Matplotlib a...

Objective: Extend the exploration notebook to plot a bar chart of class distribution using Matplotlib and save it to `ml/evaluation/class_distribution.png`.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/evaluation/class_distribution.png`

Detailed Requirements

- Extend the exploration notebook to plot a bar chart of class distribution using Matplotlib and save it to `ml/evaluation/class_distribution.png`.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.

- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 9 — Add a script ml/dataset/filter_classes.py that copies only the 8 target classes (Tomato He...

Objective: Add a script `ml/dataset/filter_classes.py` that copies only the 8 target classes (Tomato Healthy/Early Blight/Late Blight, Potato Healthy/Early Blight/Late Blight, Pepper Healthy/Bacterial Spot) from the raw dataset into `ml/dataset/filtered/`.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/filter_classes.py`, `ml/dataset/filtered/`

Detailed Requirements

- Add a script `ml/dataset/filter_classes.py` that copies only the 8 target classes (Tomato Healthy/Early Blight/Late Blight, Potato Healthy/Early Blight/Late Blight, Pepper Healthy/Bacterial Spot) from the raw dataset into `ml/dataset/filtered/`.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 10 — Write a corrupted-image and duplicate-image checker script ml/dataset/validate_images.py u...

Objective: Write a corrupted-image and duplicate-image checker script `ml/dataset/validate_images.py` using PIL and hashlib, and generate a validation report.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/validate_images.py`

Detailed Requirements

- Write a corrupted-image and duplicate-image checker script `ml/dataset/validate_images.py` using PIL and hashlib, and generate a validation report.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 11 — Add a script ml/dataset/image_stats.py that reports min/max/average image dimensions per c...

Objective: Add a script `ml/dataset/image_stats.py` that reports min/max/average image dimensions per class.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/image_stats.py`

Detailed Requirements

- Add a script `ml/dataset/image_stats.py` that reports min/max/average image dimensions per class.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 12 — Document the discovered class imbalance in ml/dataset/CLASS_DISTRIBUTION.md, including cou...

Objective: Document the discovered class imbalance in `ml/dataset/CLASS_DISTRIBUTION.md`, including counts per class.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/CLASS_DISTRIBUTION.md`

Detailed Requirements

- Document the discovered class imbalance in `ml/dataset/CLASS_DISTRIBUTION.md`, including counts per class.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 13 — Write ml/dataset/split_dataset.py that performs a stratified 70/15/15 train/val/test split...

Objective: Write `ml/dataset/split_dataset.py` that performs a stratified 70/15/15 train/val/test split and writes the results into `ml/dataset/split/{train,val,test}/<class>/`.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/split_dataset.py`, `ml/dataset/split/{train,val,test}/<class>/`

Detailed Requirements

- Write `ml/dataset/split_dataset.py` that performs a stratified 70/15/15 train/val/test split and writes the results into `ml/dataset/split/{train,val,test}/<class>/`.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 14 — Add a sanity-check notebook cell that displays 5 random sample images per class to visually ...

Objective: Add a sanity-check notebook cell that displays 5 random sample images per class to visually confirm labels are correct.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add a sanity-check notebook cell that displays 5 random sample images per class to visually confirm labels are correct.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.

- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

PROMPT 15 — Write a small ml/dataset/README.md documenting dataset source, license, class list, and fo...

Objective: Write a small `ml/dataset/README.md` documenting dataset source, license, class list, and folder layout.

Why This Matters: Everything downstream — preprocessing, training, evaluation — depends on the dataset being correctly downloaded, labelled, profiled, and split before any model code is written. Mistakes here (wrong class folders, leakage between train/test) silently corrupt every later accuracy number.

Primary File(s): `ml/dataset/README.md`

Detailed Requirements

- Write a small `ml/dataset/README.md` documenting dataset source, license, class list, and folder layout.
- Log counts/row numbers to the console or a report file so the step is auditable, not a silent pass/fail.
- Fail loudly (raise an exception / print a clear error) rather than silently skipping a missing or malformed file.
- Keep raw downloaded data separate from any filtered/cleaned copies so the original can always be re-derived.
- Make any randomness (shuffling, splitting) seeded and reproducible.

Technical Specification

Operates on the PlantVillage dataset folder layout (`<crop>___<condition>/` per class). Target scope: 8 classes — Tomato (Healthy/Early Blight/Late Blight), Potato (Healthy/Early Blight/Late Blight), Pepper (Healthy/Bacterial Spot). Use Pillow/OpenCV for any image I/O and Pandas for any tabular summary.

Acceptance Criteria

- ☐ Script runs end-to-end on the full dataset without manual intervention.
- ☐ Output (file, folder, or printed report) matches what the objective describes.
- ☐ Class counts/labels in the output are human-verified against a few known sample images.

Edge Cases & Notes

- Handle at least one corrupted or non-image file in the source folder gracefully (skip + log, don't crash).
- Document any class found to be significantly imbalanced relative to the others.

1.C Pre-processing Pipeline

PROMPT 16 — Create ml/preprocessing/loader.py implementing an image loader that reads, converts to RGB...

Objective: Create `ml/preprocessing/loader.py` implementing an image loader that reads, converts to RGB, resizes to 224x224, and normalizes pixel values to [0,1].

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/loader.py`

Detailed Requirements

- Create `ml/preprocessing/loader.py` implementing an image loader that reads, converts to RGB, resizes to 224x224, and normalizes pixel values to [0,1].
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 17 — Create ml/preprocessing/augmentation.py defining a Keras augmentation pipeline with rotati...

Objective: Create `ml/preprocessing/augmentation.py` defining a Keras augmentation pipeline with rotation, horizontal flip, zoom, translation, and brightness adjustment.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/augmentation.py`

Detailed Requirements

- Create `ml/preprocessing/augmentation.py` defining a Keras augmentation pipeline with rotation, horizontal flip, zoom, translation, and brightness adjustment.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 18 — Write a notebook ml/notebooks/02_preprocessing_test.ipynb that visually verifies preproces...

Objective: Write a notebook `ml/notebooks/02_preprocessing_test.ipynb` that visually verifies preprocessing output on 6 sample images (before/after).

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/notebooks/02_preprocessing_test.ipynb`

Detailed Requirements

- Write a notebook `ml/notebooks/02_preprocessing_test.ipynb` that visually verifies preprocessing output on 6 sample images (before/after).
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 19 — Write a notebook cell that visually verifies augmentation output by plotting 8 augmented var...

Objective: Write a notebook cell that visually verifies augmentation output by plotting 8 augmented variants of one sample image.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Write a notebook cell that visually verifies augmentation output by plotting 8 augmented variants of one sample image.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 20 — Create ml/preprocessing/dataset_pipeline.py that builds tf.data.Dataset objects for trai...

Objective: Create `ml/preprocessing/dataset_pipeline.py` that builds `tf.data.Dataset` objects for train/val/test using `image_dataset_from_directory`.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/dataset_pipeline.py`, `tf.data.Dataset`

Detailed Requirements

- Create `ml/preprocessing/dataset_pipeline.py` that builds `tf.data.Dataset` objects for train/val/test using `image_dataset_from_directory`.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 21 — Add batching and prefetching (.batch(), .prefetch(AUTOTUNE)) configuration to the tf.da...

Objective: Add batching and prefetching (`.batch()`, `.prefetch(AUTOTUNE)`) configuration to the `tf.data.Dataset` pipeline for performance.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `.batch()`, `.prefetch(AUTOTUNE)`, `tf.data.Dataset`

Detailed Requirements

- Add batching and prefetching (`.batch()`, `.prefetch(AUTOTUNE)`) configuration to the `tf.data.Dataset` pipeline for performance.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 22 — Add a caching layer (.cache()) to the pipeline for the validation and test sets to speed u...

Objective: Add a caching layer (`.cache()`) to the pipeline for the validation and test sets to speed up repeated evaluation runs.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `.cache()`

Detailed Requirements

- Add a caching layer (`.cache()`) to the pipeline for the validation and test sets to speed up repeated evaluation runs.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.

- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 23 — Write unit tests ml/preprocessing/test_loader.py verifying output image shape and pixel-va...

Objective: Write unit tests `ml/preprocessing/test_loader.py` verifying output image shape and pixel-value range.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/test_loader.py`

Detailed Requirements

- Write unit tests `ml/preprocessing/test_loader.py` verifying output image shape and pixel-value range.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 24 — Add class-weight computation ml/preprocessing/class_weights.py using sklearn.utils.class_...

Objective: Add class-weight computation `ml/preprocessing/class_weights.py` using `sklearn.utils.class_weight` to address the discovered imbalance.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/class_weights.py`, `sklearn.utils.class_weight`

Detailed Requirements

- Add class-weight computation `ml/preprocessing/class_weights.py` using `sklearn.utils.class_weight` to address the discovered imbalance.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

PROMPT 25 — Document the full preprocessing pipeline (steps + rationale) in ml/preprocessing/README.md

Objective: Document the full preprocessing pipeline (steps + rationale) in `ml/preprocessing/README.md`.

Why This Matters: The exact preprocessing transform used at training time must be reproduced byte-for-byte at inference time in the backend, or the model will silently underperform in production. This prompt's output becomes the single source of truth both sides reference.

Primary File(s): `ml/preprocessing/README.md`

Detailed Requirements

- Document the full preprocessing pipeline (steps + rationale) in `ml/preprocessing/README.md`.
- Resize target is 224×224×3 (matching MobileNetV2/ResNet50 input expectations) unless a specific model dictates otherwise.
- Normalize pixel values consistently (either [0,1] scaling or the backbone's own `preprocess_input`, not a mix of both).
- Keep augmentation strictly inside the training pipeline — never apply random augmentation to validation/test data or to production inference images.
- Write the function so it can be imported directly by both a training script and (later) the backend's inference service without modification.

Technical Specification

Depends on: Pillow/OpenCV for image I/O, NumPy for array ops, `tf.data.Dataset` / `image_dataset_from_directory` for pipeline construction. Output tensor shape must be (224, 224, 3), dtype float32.

Acceptance Criteria

- ☐ A single sample image run through the function produces the expected output shape and value range.
- ☐ Visual inspection of a few processed/augmented images confirms no biologically nonsensical distortion.
- ☐ The function has no hidden dependency on global state (pure function where possible).

Edge Cases & Notes

- Handle grayscale or RGBA input images by converting to RGB rather than erroring.
- Handle unusually large source images (e.g. >4000px) without excessive memory use.

1.D Baseline CNN

PROMPT 26 — Create ml/training/baseline_cnn.py defining a simple CNN architecture (3-4 Conv2D+MaxPooli...

Objective: Create `ml/training/baseline_cnn.py` defining a simple CNN architecture (3-4 Conv2D+MaxPooling blocks, Dense head) using the Keras Sequential API.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/baseline_cnn.py`

Detailed Requirements

- Create `ml/training/baseline_cnn.py` defining a simple CNN architecture (3-4 Conv2D+MaxPooling blocks, Dense head) using the Keras Sequential API.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 27 — Write ml/training/train_baseline.py that compiles the baseline model (Adam optimizer, cate...

Objective: Write `ml/training/train_baseline.py` that compiles the baseline model (Adam optimizer, categorical cross-entropy) and trains it for 15 epochs on the train/val datasets.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/train_baseline.py`

Detailed Requirements

- Write `ml/training/train_baseline.py` that compiles the baseline model (Adam optimizer, categorical cross-entropy) and trains it for 15 epochs on the train/val datasets.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 28 — Add TensorBoard logging and Keras ModelCheckpoint/EarlyStopping callbacks to train_base...

Objective: Add TensorBoard logging and Keras `ModelCheckpoint`/`EarlyStopping` callbacks to `train_baseline.py`.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `train_baseline.py`

Detailed Requirements

- Add TensorBoard logging and Keras `ModelCheckpoint`/`EarlyStopping` callbacks to `train_baseline.py`.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 29 — Write ml/evaluation/plot_training_curves.py that plots training/validation accuracy and lo...

Objective: Write `ml/evaluation/plot_training_curves.py` that plots training/validation accuracy and loss curves and saves them as PNGs.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/evaluation/plot_training_curves.py`

Detailed Requirements

- Write `ml/evaluation/plot_training_curves.py` that plots training/validation accuracy and loss curves and saves them as PNGs.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 30 — Run the baseline training and save the resulting model to ml/models/baseline_cnn.keras

Objective: Run the baseline training and save the resulting model to `ml/models/baseline_cnn.keras`.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/models/baseline_cnn.keras`

Detailed Requirements

- Run the baseline training and save the resulting model to `ml/models/baseline_cnn.keras`.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 31 — Evaluate the baseline model on the test set and log accuracy/loss to ml/evaluation/baseline...

Objective: Evaluate the baseline model on the test set and log accuracy/loss to `ml/evaluation/baseline_results.json`.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/evaluation/baseline_results.json`

Detailed Requirements

- Evaluate the baseline model on the test set and log accuracy/loss to `ml/evaluation/baseline_results.json`.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 32 — Write a short analysis note ml/evaluation/baseline_analysis.md documenting observed overfi...

Objective: Write a short analysis note `ml/evaluation/baseline_analysis.md` documenting observed overfitting/underfitting behavior from the curves.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/evaluation/baseline_analysis.md`

Detailed Requirements

- Write a short analysis note `ml/evaluation/baseline_analysis.md` documenting observed overfitting/underfitting behavior from the curves.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 33 — Add dropout and L2 regularization to a second baseline variant ml/training/baseline_cnn_v2....

Objective: Add dropout and L2 regularization to a second baseline variant `ml/training/baseline_cnn_v2.py` if overfitting was observed, and re-train.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/baseline_cnn_v2.py`

Detailed Requirements

- Add dropout and L2 regularization to a second baseline variant `ml/training/baseline_cnn_v2.py` if overfitting was observed, and re-train.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

1.E Transfer Learning

PROMPT 34 — Create ml/training/transfer_mobilenetv2.py that loads MobileNetV2 with ImageNet weights, f...

Objective: Create `ml/training/transfer_mobilenetv2.py` that loads MobileNetV2 with ImageNet weights, freezes the base, and attaches a new classification head for 8 classes.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/transfer_mobilenetv2.py`

Detailed Requirements

- Create `ml/training/transfer_mobilenetv2.py` that loads MobileNetV2 with ImageNet weights, freezes the base, and attaches a new classification head for 8 classes.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 35 — Write ml/training/train_transfer.py to train the new classification head for 10 epochs whi...

Objective: Write `ml/training/train_transfer.py` to train the new classification head for 10 epochs while the base is frozen.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/train_transfer.py`

Detailed Requirements

- Write `ml/training/train_transfer.py` to train the new classification head for 10 epochs while the base is frozen.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 36 — Extend train_transfer.py with a fine-tuning phase

Objective: Extend `train_transfer.py` with a fine-tuning phase: unfreeze the top N layers of MobileNetV2 and continue training at a lower learning rate for 10 more epochs.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `train_transfer.py`

Detailed Requirements

- Extend `train_transfer.py` with a fine-tuning phase: unfreeze the top N layers of MobileNetV2 and continue training at a lower learning rate for 10 more epochs.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 37 — Save the trained transfer-learning model to ml/models/mobilenetv2_final.keras

Objective: Save the trained transfer-learning model to `ml/models/mobilenetv2_final.keras`.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/models/mobilenetv2_final.keras`

Detailed Requirements

- Save the trained transfer-learning model to `ml/models/mobilenetv2_final.keras`.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 38 — Create ml/training/transfer_resnet50.py implementing the same transfer-learning approach u...

Objective: Create `ml/training/transfer_resnet50.py` implementing the same transfer-learning approach using ResNet50 as a comparison architecture.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/training/transfer_resnet50.py`

Detailed Requirements

- Create `ml/training/transfer_resnet50.py` implementing the same transfer-learning approach using ResNet50 as a comparison architecture.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 39 — Train and save the ResNet50 variant to ml/models/resnet50_final.keras

Objective: Train and save the ResNet50 variant to `ml/models/resnet50_final.keras`.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/models/resnet50_final.keras`

Detailed Requirements

- Train and save the ResNet50 variant to `ml/models/resnet50_final.keras`.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.

- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 40 — Write ml/evaluation/compare_models.py that loads baseline, MobileNetV2, and ResNet50 model…

Objective: Write `ml/evaluation/compare_models.py` that loads baseline, MobileNetV2, and ResNet50 models and compares their test-set accuracy, inference time, and file size in a single table.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/evaluation/compare_models.py`

Detailed Requirements

- Write `ml/evaluation/compare_models.py` that loads baseline, MobileNetV2, and ResNet50 models and compares their test-set accuracy, inference time, and file size in a single table.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 41 — Based on the comparison, write ml/evaluation/MODEL_SELECTION.md documenting which model wa...

Objective: Based on the comparison, write `ml/evaluation/MODEL_SELECTION.md` documenting which model was chosen as final and why.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/evaluation/MODEL_SELECTION.md`

Detailed Requirements

- Based on the comparison, write `ml/evaluation/MODEL_SELECTION.md` documenting which model was chosen as final and why.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 42 — Copy the selected final model to ml/models/crop_disease_model.keras (the production filena...

Objective: Copy the selected final model to `ml/models/crop_disease_model.keras` (the production filename referenced by the backend).

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/models/crop_disease_model.keras`

Detailed Requirements

- Copy the selected final model to `ml/models/crop_disease_model.keras` (the production filename referenced by the backend).
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

PROMPT 43 — Generate ml/models/class_names.json mapping numeric class indices to human-readable crop/d...

Objective: Generate `ml/models/class_names.json` mapping numeric class indices to human-readable crop/disease labels, matching the final model's output order.

Why This Matters: This is the core modelling step the entire product depends on. The architecture, optimizer, and callback choices made here directly determine the accuracy numbers reported in the final evaluation chapter and the confidence the backend can place in its predictions.

Primary File(s): `ml/models/class_names.json`

Detailed Requirements

- Generate `ml/models/class_names.json` mapping numeric class indices to human-readable crop/disease labels, matching the final model's output order.
- Use categorical cross-entropy loss and the Adam optimizer unless a specific experiment calls for otherwise.
- Log training/validation accuracy and loss every epoch (via Keras `History` or TensorBoard) — do not discard this data.
- Use `ModelCheckpoint` to save the best-performing epoch's weights, not just the final epoch.
- Use `EarlyStopping` on validation loss to avoid wasting compute once the model stops improving.
- Save the finished model in `.keras` format to the exact path specified in the project's file structure.

Technical Specification

Framework: TensorFlow/Keras. For transfer-learning prompts: load the named backbone (MobileNetV2 or ResNet50) with `weights='imagenet', include_top=False`, add `GlobalAveragePooling2D` + `Dense(8, activation='softmax')` head. For fine-tuning prompts: unfreeze the top layers only, recompile with a low learning rate (e.g. 1e-5), and continue training.

Acceptance Criteria

- ☐ Training completes without NaN loss or crashing.
- ☐ The saved `.keras` file loads successfully and produces predictions of shape (batch, 8).
- ☐ Training/validation accuracy and loss curves are saved as artifacts for the report.

Edge Cases & Notes

- If training accuracy diverges sharply above validation accuracy, flag this as overfitting in the output report rather than ignoring it.
- If GPU is unavailable, confirm the script still runs (slower) on CPU without errors.

1.F Model Evaluation

PROMPT 44 — Write ml/evaluation/evaluate.py computing accuracy, precision, recall, and F1-score per cl...

Objective: Write `ml/evaluation/evaluate.py` computing accuracy, precision, recall, and F1-score per class using scikit-learn's `classification_report`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/evaluate.py`

Detailed Requirements

- Write `ml/evaluation/evaluate.py` computing accuracy, precision, recall, and F1-score per class using scikit-learn's `classification_report`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 45 — Write ml/evaluation/confusion_matrix.py generating and plotting a confusion matrix heatmap...

Objective: Write `ml/evaluation/confusion_matrix.py` generating and plotting a confusion matrix heatmap for the final model on the test set.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/confusion_matrix.py`

Detailed Requirements

- Write `ml/evaluation/confusion_matrix.py` generating and plotting a confusion matrix heatmap for the final model on the test set.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 46 — Save all evaluation metrics to ml/evaluation/final_model_metrics.json and ml/evaluation/f...

Objective: Save all evaluation metrics to `ml/evaluation/final_model_metrics.json` and `ml/evaluation/final_model_report.md`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/final_model_metrics.json`, `ml/evaluation/final_model_report.md`

Detailed Requirements

- Save all evaluation metrics to `ml/evaluation/final_model_metrics.json` and `ml/evaluation/final_model_report.md`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 47 — Write ml/evaluation/confidence_threshold_experiment.py that sweeps threshold values (e

Objective: Write `ml/evaluation/confidence_threshold_experiment.py` that sweeps threshold values (e.g. 0.5 to 0.9) and reports how many test predictions would be flagged as "uncertain" at each threshold.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/confidence_threshold_experiment.py`

Detailed Requirements

- Write `ml/evaluation/confidence_threshold_experiment.py` that sweeps threshold values (e.g. 0.5 to 0.9) and reports how many test predictions would be flagged as "uncertain" at each threshold.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.

- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 48 — Based on the sweep, document the chosen production confidence threshold and rationale in ml...

Objective: Based on the sweep, document the chosen production confidence threshold and rationale in `ml/evaluation/CONFIDENCE_THRESHOLD.md`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/CONFIDENCE_THRESHOLD.md`

Detailed Requirements

- Based on the sweep, document the chosen production confidence threshold and rationale in `ml/evaluation/CONFIDENCE_THRESHOLD.md`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 49 — Write ml/evaluation/error_analysis.py that saves the top 10 most-confident wrong predictio...

Objective: Write `ml/evaluation/error_analysis.py` that saves the top 10 most-confident wrong predictions as images for manual review.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/error_analysis.py`

Detailed Requirements

- Write `ml/evaluation/error_analysis.py` that saves the top 10 most-confident wrong predictions as images for manual review.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 50 — Summarize key findings (best model, accuracy, confusion patterns) in ml/evaluation/SUMMARY.md

Objective: Summarize key findings (best model, accuracy, confusion patterns) in `ml/evaluation/SUMMARY.md`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/SUMMARY.md`

Detailed Requirements

- Summarize key findings (best model, accuracy, confusion patterns) in `ml/evaluation/SUMMARY.md`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

1.G Real-World Testing

PROMPT 51 — Collect (or simulate collecting) 30–50 independently sourced real-world leaf photographs and...

Objective: Collect (or simulate collecting) 30–50 independently sourced real-world leaf photographs and organize them into `ml/dataset/real_world_sample/<class>/`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/dataset/real_world_sample/<class>/`

Detailed Requirements

- Collect (or simulate collecting) 30–50 independently sourced real-world leaf photographs and organize them into `ml/dataset/real_world_sample/<class>/`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 52 — Write ml/evaluation/real_world_eval.py that runs the final model against the real-world sa...

Objective: Write `ml/evaluation/real_world_eval.py` that runs the final model against the real-world sample and reports accuracy.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/real_world_eval.py`

Detailed Requirements

- Write `ml/evaluation/real_world_eval.py` that runs the final model against the real-world sample and reports accuracy.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 53 — Write ml/evaluation/dataset_vs_realworld.py producing a side-by-side comparison table of t...

Objective: Write `ml/evaluation/dataset_vs_realworld.py` producing a side-by-side comparison table of test-set accuracy vs. real-world accuracy.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/dataset_vs_realworld.py`

Detailed Requirements

- Write `ml/evaluation/dataset_vs_realworld.py` producing a side-by-side comparison table of test-set accuracy vs. real-world accuracy.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 54 — Document the real-world performance gap and likely causes (lighting, background, camera qual...

Objective: Document the real-world performance gap and likely causes (lighting, background, camera quality) in `ml/evaluation/REAL_WORLD_FINDINGS.md`.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/evaluation/REAL_WORLD_FINDINGS.md`

Detailed Requirements

- Document the real-world performance gap and likely causes (lighting, background, camera quality) in `ml/evaluation/REAL_WORLD_FINDINGS.md`.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.

- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.

PROMPT 55 — Finalize and freeze ml/models/crop_disease_model.keras and ml/models/class_names.json as...

Objective: Finalize and freeze `ml/models/crop_disease_model.keras` and `ml/models/class_names.json` as the production artifacts to be handed to the backend team.

Why This Matters: A model is only as trustworthy as its evaluation. This prompt's output is what justifies every accuracy claim written in the final report and what determines the production confidence threshold used by the backend's `/predict` endpoint.

Primary File(s): `ml/models/crop_disease_model.keras`, `ml/models/class_names.json`

Detailed Requirements

- Finalize and freeze `ml/models/crop_disease_model.keras` and `ml/models/class_names.json` as the production artifacts to be handed to the backend team.
- Evaluate strictly on the held-out test split (or the independently collected real-world sample where specified) — never on data used for training or tuning.
- Report metrics per class as well as overall — a single overall accuracy number hides class-level weaknesses.
- Save all generated numbers/plots to files (not just printed to console) so they can be embedded in the final report.
- Use scikit-learn (`classification_report`, `confusion_matrix`) rather than hand-rolled metric calculations.

Technical Specification

Inputs: the final saved `.keras` model + `class_names.json` + the relevant test/real-world dataset split. Outputs: accuracy, precision, recall, F1 per class (scikit-learn `classification_report`), a confusion-matrix heatmap image, and a JSON/Markdown summary file.

Acceptance Criteria

- ☐ Generated report file contains per-class precision/recall/F1 and an overall accuracy figure.
- ☐ Confusion matrix image is saved and legible (axis-labelled with class names, not raw indices).
- ☐ Numbers in the report are the actual measured output of this run, not placeholder/assumed values.

Edge Cases & Notes

- If any class has zero test examples predicted correctly, call this out explicitly rather than letting it blend into the average.
- Note any meaningful gap between dataset-test accuracy and real-world-sample accuracy if both are being compared in this prompt.
